



COMANDOS REACT - GUIA COMPLETO



COMANDOS BÁSICOS DO CLI

Criação de Projetos

```
bash
```

```
# Criar novo projeto React com Create React App
```

```
npx create-react-app meu-projeto
```

```
cd meu-projeto
```

```
npm start
```

```
# Criar projeto React com Vite (Mais rápido)
```

```
npm create vite@latest meu-projeto-react -- --template react
```

```
cd meu-projeto-react
```

```
npm install
```

```
npm run dev
```

```
# Criar projeto React com TypeScript
```

```
npx create-react-app meu-projeto --template typescript
```

```
npm create vite@latest meu-projeto -- --template react-ts
```

Execução e Build

```
bash
```

```
# Desenvolvimento
```

```
npm start # Create React App
```

```
npm run dev # Vite
```

```
# Build de produção
```

```
npm run build
```

```
# Servir build de produção localmente
```

```
npm install -g serve
```

```
serve -s build
```

```
# Executar testes
npm test
```

```
npm run test:watch # Modo watch
```



COMANDOS PARA COMPONENTES E ESTRUTURA

Com Create React App

```
bash
```

```
# Não possui gerador built-in, mas pode usar:
npx generate-react-cli component MeuComponente
```

```
# Ou instalar geradores alternativos
npm install -g react-cli
```

```
react-cli component Header
```

Com Vite + Plugins

```
bash
```

```
# Instalar gerador de componentes
npm install -g plop
```

```
# Criar template plopfile.js para gerar componentes
```

Estrutura Manual Recomendada

```
bash
```

```
# Estrutura de pastas típica
```

```
src/  
  components/  
    Button/  
      Button.jsx  
      Button.module.css  
      index.js  
  pages/  
    Home/  
      Home.jsx  
  hooks/  
  services/
```

```
utils/
```



GESTÃO DE DEPENDÊNCIAS

Bibliotecas Essenciais

```
bash
```

```
# Roteamento
```

```
npm install react-router-dom
```

```
npm install @types/react-router-dom # Para TypeScript
```

```
# Gerenciamento de Estado
```

```
npm install redux @reduxjs/toolkit react-redux
```

```
npm install zustand # Estado simples
```

```
npm install recoil # Estado atômico
```

```
# Estilização
```

```
npm install styled-components
```

```
npm install @emotion/react @emotion/styled
```

```
npm install tailwindcss
```

```
# HTTP Requests
```

```
npm install axios
```

```
npm install react-query # Cache e gerenciamento
```

```
npm install swr # Data fetching
```

```
# Formulários
```

```
npm install react-hook-form
```

```
npm install formik yup
```

```
# UI Components
```

```
npm install @mui/material @emotion/react @emotion/styled
```

```
npm install antd
```

```
npm install react-bootstrap bootstrap
```

Desenvolvimento

```
bash
```

```
# Ferramentas de desenvolvimento
```

```
npm install -D eslint @eslint/create-config
```

```
npm install -D prettier
```

```
npm install -D husky lint-staged
```

```
# Bundle Analyzer
```

```
npm install -D webpack-bundle-analyzer
```



COMANDOS DE DESENVOLVIMENTO

Desenvolvimento com Features

```
bash
```

```
# Desenvolvimento com diferentes ambientes
```

```
npm start # Ambiente dev
```

```
npm run build # Build produção
```

```
# Com variáveis de ambiente
```

```
REACT_APP_API_URL=http://localhost:3001 npm start
```

```
# Build com análise
```

```
npm run build -- --profile
```

```
npm install -g serve
```

```
serve -s build -l 3000
```

Testes

```
bash
```

```
# Executar testes
```

```
npm test
```

```
npm test -- --coverage --watchAll=false
```

```
npm run test:coverage
```

```
# Testing Library
```

```
npm install -D @testing-library/react @testing-library/jest-dom
```

```
npm install -D @testing-library/user-event
```

```
# e2e Testing
```

```
npm install -D cypress
```

```
npx cypress open
```



CONFIGURAÇÕES AVANÇADAS

ESLint + Prettier

```
bash
```

```
# Configurar ESLint
```

```
npx eslint --init
```

```
# Configurar Prettier
```

```
npm install --save-dev prettier
```

```
echo {}> .prettierrc.json
```

```
# Configurar scripts no package.json
```

```
{  
  "scripts": {  
    "lint": "eslint src --ext .js,.jsx,.ts,.tsx",  
    "lint:fix": "npm run lint -- --fix",  
    "format": "prettier --write src/"  
  }  
}
```

```
}
```

Git Hooks

```
bash
```

```
# Configurar Husky
npm install --save-dev husky lint-staged
npx husky init
npm pkg set scripts.prepare="husky install"

# Adicionar pre-commit hook
npx husky add .husky/pre-commit "npx lint-staged"

# Configurar Lint-staged no package.json
{
  "lint-staged": {
    "*.{js,jsx,ts,tsx}": [
      "eslint --fix",
      "prettier --write"
    ]
  }
}
```



COMANDOS PARA DEPLOY

Build Otimizado

```
bash
```

```
# Build de produção
npm run build

# Analisar bundle
npm install -g source-map-explorer
npx source-map-explorer 'build/static/js/*.js'

# Ou com webpack-bundle-analyzer
```

```
npx webpack-bundle-analyzer build/static/js/*.js
```

Deploy em Diferentes Plataformas

```
bash
```

```
# Netlify
```

```
npm run build
```

```
# Fazer upload da pasta 'build'
```

```
# Vercel
```

```
npm install -g vercel
```

```
vercel --prod
```

```
# GitHub Pages
```

```
npm install --save-dev gh-pages
```

```
# Adicionar no package.json:
```

```
{  
  "homepage": "https://meuusuario.github.io/meu-repositorio",  
  "scripts": {  
    "predeploy": "npm run build",  
    "deploy": "gh-pages -d build"  
  }  
}
```

```
npm run deploy
```



COMANDOS PARA REACT NATIVE

React Native CLI

```
bash
```

```
# Instalar CLI
```

```
npm install -g @react-native-community/cli
```

```
# Criar novo projeto
```

```
npx react-native init MeuProjeto
```

```
# Executar
```

```
npx react-native run-android
```

```
npx react-native run-ios
```

```
# Metro Bundler
```

```
npx react-native start
```

```
# Executar em dispositivo físico
```

```
npx react-native run-android --deviceId=your_device_id
```

Expo

```
bash
```

```
# Instalar Expo CLI  
npm install -g @expo/cli
```

```
# Criar projeto  
npx create-expo-app MinhaApp  
cd MinhaApp  
npx expo start
```

```
# Build  
npx expo build:android
```

```
npx expo build:ios
```



COMANDOS DE ATUALIZAÇÃO

Atualizar React

```
bash
```

```
# Ver versões instaladas  
npm list react react-dom
```

```
# Atualizar para última versão  
npm update react react-dom
```

```
# Atualizar Create React App  
npm install -g create-react-app
```

```
# Migrar para versões mais recentes  
npx npm-check-updates -u
```

```
npm install
```


Atualizar Dependências

```
bash
```

```
# Ver dependências desatualizadas  
npm outdated
```

```
# Atualizar todas as dependências  
npx npm-check-updates  
npx npm-check-updates -u  
npm install
```

```
# Atualizar dependência específica
```

```
npm install react@latest
```



COMANDOS ÚTEIS DO DIA A DIA

Desenvolvimento Rápido

```
bash
```

```
# Desenvolvimento com backend proxy  
# Adicionar no package.json:  
{  
  "proxy": "http://localhost:3001"  
}
```

```
# Ambiente de desenvolvimento com HTTPS  
HTTPS=true npm start
```

```
# Porta personalizada  
PORT=3005 npm start
```

```
# Abrir no navegador automaticamente
```

```
BROWSER=chrome npm start
```

Debug e Análise

```
bash
```

```
# Analisar performance
```

```
npm run build
```

```
npx lighthouse http://localhost:3000 --view
```

```
# Debug com React DevTools
```

```
# Instalar extensão no navegador
```

```
# Ver tamanho das dependências
```

```
npm ls --depth=0
```

```
npx depcheck
```



EXEMPLO DE FLUXO DE TRABALHO COMPLETO

```
bash
```

```
# 1. Criar projeto com Vite
```

```
npm create vite@latest meu-app-react -- --template react
```

```
cd meu-app-react
```

```
npm install
```

```
# 2. Instalar dependências essenciais
```

```
npm install react-router-dom axios styled-components
```

```
npm install -D eslint prettier husky
```

```
# 3. Configurar ESLint
```

```
npx eslint --init
```

```
# 4. Desenvolver
```

```
npm run dev
```

```
# 5. Gerar estrutura de pastas manualmente
```

```
mkdir -p src/{components,pages,hooks,services,utils}
```

```
# 6. Adicionar componentes manualmente
```

```
# Criar: src/components/Button/Button.jsx
```

```
# Criar: src/components/Button/index.js
```

```
# 7. Testar
```

```
npm run test
```

```
# 8. Build final
```

```
npm run build
```

9. Deploy

```
npm run deploy
```

SCRIPTS RECOMENDADOS NO package.json

```
json
```

```
{
  "scripts": {
    "dev": "vite",
    "start": "react-scripts start",
    "build": "vite build",
    "preview": "vite preview",
    "test": "react-scripts test",
    "test:coverage": "npm test -- --coverage --watchAll=false",
    "lint": "eslint src --ext .js,.jsx,.ts,.tsx",
    "lint:fix": "npm run lint -- --fix",
    "format": "prettier --write src/",
    "analyze": "source-map-explorer build/static/js/*.js",
    "predeploy": "npm run build",
    "deploy": "gh-pages -d build"
  }
}
```

Este guia cobre todos os comandos essenciais para desenvolvimento

React profissional! 🚀